

Unidad Didáctica

Lenguaje Algorítmico, Unidad II

Índice de Contenido

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	5
Tipos de Datos Elementales.....	5
Tipos de Datos Simples o Básicos	6
Elementos Básicos	7
Identificador	7
Variables	8
Constantes	9
Operadores y Expresiones	9
Operadores Aritméticos, relacionales y lógicos.....	9
Expresiones lógicas	11
Instrucciones Entrada / Salida.....	12
Estructura general de un Algoritmo	12
Estructuras Básicas.....	14
Los comentarios.....	14
Bloque de asignación.....	15
La estructura secuencial	15
La estructura alternativa.....	15
La estructura iterativa	16
Bibliografía y Webgrafía	17
Glosario.....	17

Introducción



En esta segunda unidad, nos adentraremos en el fascinante mundo del lenguaje algorítmico, un conjunto de reglas y estructuras que nos permiten resolver problemas de manera eficiente y sistemática. Nuestro objetivo principal es dotarte de las herramientas

necesarias para que puedas estructurar algoritmos básicos y diseñar soluciones efectivas para una amplia variedad de situaciones.

Durante esta unidad, aprenderás a manejar adecuadamente los términos referentes a los tipos de datos elementales, comprendiendo tanto los tipos simples predefinidos como aquellos definidos por el programador. Además, exploraremos los elementos fundamentales de cualquier algoritmo, desde identificadores y variables hasta constantes y operadores, incluyendo tanto los aritméticos como los relacionales y lógicos.

No solo nos enfocaremos en la teoría, sino que también te proporcionaremos las herramientas prácticas necesarias para que puedas diseñar algoritmos desde la conceptualización de un problema, utilizando la terminología algorítmica de manera efectiva. Asimismo, fomentaremos un compromiso ético tanto con el trabajo individual como con el trabajo en equipo, reconociendo la importancia de la integridad en el ámbito de la programación y la resolución de problemas.

A lo largo de esta unidad, exploraremos la estructura general de un algoritmo, así como las diferentes estructuras básicas que lo componen, desde los comentarios

hasta las estructuras secuenciales, alternativas e iterativas. Estas herramientas te permitirán no solo comprender el funcionamiento interno de un algoritmo, sino también desarrollar la habilidad para crear soluciones eficientes y elegantes para una amplia gama de problemas.

¡Prepárate para sumergirte en el emocionante mundo del lenguaje algorítmico y descubrir el poder que reside en la capacidad de resolver problemas de manera sistemática y estructurada!

Tipos de Datos Elementales

los tipos de datos elementales pueden clasificarse en cualitativos y cuantitativos, cada uno con sus respectivas subdivisiones.

Tipos de Datos Cualitativos: Los datos cualitativos describen cualidades o atributos que no pueden ser cuantificados numéricamente de forma directa. A su vez, se subdividen en:

- Nominales: Representan categorías o clases que no tienen un orden específico. En PSeInt, podríamos utilizar este tipo de dato para representar, por ejemplo, el género de una persona o el tipo de animal.
- Ordinales: Indican un orden o jerarquía entre las categorías. Aunque no se puede realizar operaciones aritméticas con ellos, sí se puede establecer un orden. Por ejemplo, podríamos utilizar este tipo de dato para representar el nivel de satisfacción de un cliente (bajo, medio, alto).

Tipos de Datos Cuantitativos: Los datos cuantitativos, por otro lado, representan cantidades numéricas y se pueden subdividir en:

- Discretos: Representan valores que son contables y separados por unidades definidas. Por ejemplo, el número de estudiantes en un salón de clases o el número de libros en una biblioteca.
- Continuos: Representan valores que pueden tomar cualquier valor dentro de un rango determinado. En contraste con los datos discretos, los datos continuos pueden tener infinitas posibilidades dentro de un intervalo. Por ejemplo, la altura de una persona o el peso de un objeto.

Tipos de Datos Simples o Básicos

Tipos de Datos Predefinidos

Los tipos de datos simples o básicos son aquellos que están predefinidos en el lenguaje de programación y que representan valores individuales. En PSeInt, algunos ejemplos comunes de tipos de datos simples son:

- **Entero (int):** Este tipo de dato se utiliza para almacenar números enteros, es decir, números sin parte decimal. Por ejemplo: `Variable: entero
edad = 25`
- **Real (float):** Permite almacenar números con parte decimal. Por ejemplo: `Variable: real precio = 15.99`
- **Carácter (char):** Este tipo de dato se utiliza para representar caracteres individuales, como letras, números o símbolos. Por ejemplo: `Variable: caracter letra = 'A'`
- **Booleano (bool):** Representa un valor lógico que puede ser verdadero (true) o falso (false). Este tipo de dato es esencial para tomar decisiones basadas en condiciones lógicas. Por ejemplo: `Variable: booleano
es_mayor = VERDADERO`

Tipos de Datos Definidos por el Programador

Además de los tipos de datos simples, en PSeInt también podemos definir nuestros propios tipos de datos. Esto nos permite organizar la información de manera más adecuada según nuestras necesidades. Por ejemplo, podríamos definir un tipo de dato para representar una fecha, una persona o cualquier otra entidad que sea relevante para nuestro programa.

Veamos un ejemplo sencillo de cómo podríamos definir un tipo de dato en PSeInt. Supongamos que queremos representar una persona con su nombre y su edad. Podríamos definir un tipo de dato llamado "Persona" de la siguiente manera:

```
Tipo: Persona
    nombre: cadena
    edad: entero
FinTipo
```

Ahora, podemos declarar variables de tipo Persona y acceder a sus atributos de la siguiente manera:

```
Variable: Persona persona1
persona1.nombre = "Juan"
persona1.edad = 30
```

En resumen, en PSeInt contamos con una variedad de tipos de datos simples predefinidos, como enteros, reales y caracteres, así como la capacidad de definir nuestros propios tipos de datos según nuestras necesidades específicas. Esta flexibilidad nos permite trabajar con distintos tipos de información de manera eficiente y organizada en nuestros programas.

Elementos Básicos

En PSeInt, como en cualquier lenguaje de programación, es fundamental comprender los elementos básicos que conforman un programa. Estos elementos son esenciales para la correcta escritura y comprensión de nuestros algoritmos. En esta sección, exploraremos los conceptos de identificador, variables y constantes.

Identificador

Un identificador en PSeInt es simplemente un nombre que se utiliza para identificar y hacer referencia a una variable, constante, función o cualquier otro elemento en nuestro programa. Es importante elegir identificadores significativos y descriptivos que ayuden a comprender el propósito de cada elemento en el código.

Por ejemplo, podemos tener un identificador como "edad" para representar la edad de una persona en nuestro programa: `Variable: entero edad`

Aunque no es una regla propia de pseint usaremos algunas normas que son convencionalismo internacional para nombrar identificadores,

- Todos los identificadores empiezan con minúscula (a excepción de clases y constantes)
- Las constantes van siempre con mayúsculas (toda la palabra)
- No se usan espacios, en su lugar se usa snake_case para variables y constantes, y se usa camelCase para funciones / métodos / subprogramas

Variables

Las variables son espacios de memoria reservados para almacenar datos que pueden cambiar durante la ejecución del programa. Estos datos pueden ser de diferentes tipos, como enteros, reales, caracteres o booleanos. Para declarar una variable en PSeInt, utilizamos la palabra clave "Variable", seguida del tipo de dato y el identificador.

Por ejemplo, para declarar una variable llamada "cantidad" que almacene un número entero, podemos hacer lo siguiente: `Variable: entero cantidad`

Una vez declarada la variable, podemos asignarle un valor utilizando el operador de asignación ("="). Por ejemplo: `cantidad = 10`

Constantes

A diferencia de las variables, las constantes son valores que no cambian durante la ejecución del programa. Son útiles para representar valores que son conocidos y fijos. En PSeInt, declaramos constantes utilizando la palabra clave "Constante", seguida del tipo de dato, el identificador y el valor asignado.

Por ejemplo, para declarar una constante llamada "PI" que almacene el valor de pi (aproximadamente 3.14159), podemos hacer lo siguiente: `Constante: real PI = 3.14159`

Al utilizar constantes, facilitamos la comprensión del código y evitamos errores causados por la modificación accidental de valores fijos.

En resumen, los identificadores, variables y constantes son elementos básicos en la construcción de algoritmos en PSeInt. Comprender su uso y aplicación nos permite escribir programas más claros, legibles y eficientes.

Operadores y Expresiones

Los operadores y expresiones son herramientas fundamentales en PSeInt que nos permiten realizar cálculos, comparaciones y tomar decisiones en nuestros programas. En esta sección, exploraremos los diferentes tipos de operadores y cómo utilizar expresiones en PSeInt, así como las instrucciones de entrada y salida para interactuar con el usuario.

Operadores Aritméticos, relacionales y lógicos

En PSeInt, los operadores se dividen en tres categorías principales: aritméticos, relacionales y lógicos.

Operadores Aritméticos: Estos operadores se utilizan para realizar operaciones matemáticas básicas, como la suma, la resta, la multiplicación y la división. Algunos ejemplos de operadores aritméticos en PSeInt son:

```
suma = num1 + num2;  
resta = num1 - num2;  
multiplicacion = num1 * num2;  
division = num1 / num2;
```

Operadores Relacionales: Estos operadores se utilizan para comparar valores y determinar la relación entre ellos. Algunos ejemplos de operadores relacionales en PSeInt son:

```
es_mayor = num1 > num2;  
es_igual = num1 == num2;  
es_diferente = num1 != num2;
```

Operadores Lógicos: Estos operadores se utilizan para combinar expresiones lógicas y realizar operaciones booleanas. Algunos ejemplos de operadores lógicos en PSeInt son:

```
y_logico = condicion1 Y condicion2;  
o_logico = condicion1 O condicion2;  
negacion = NO condición;
```

Operador	Descripción	Ejemplo
+	Suma dos operandos	resultado = num1 + num2;
-	Resta dos operandos	resultado = num1 - num2;
*	Multiplica dos operandos	resultado = num1 * num2;
/	Divide el primer operando por el segundo	resultado = num1 / num2;
^	Eleva el primer operando a la potencia del segundo	resultado = base ^ exponente,
mod	Devuelve el residuo de la división	resultado = num1 mod num2;
=	Asigna el valor del segundo operando al primero	variable = valor;
>	Compara si el primer operando es mayor que el segundo	es_mayor = num1 > num2;
<	Compara si el primer operando es menor que el segundo	es_menor = num1 < num2;
==	Compara si los dos operandos son iguales	es_igual = num1 == num2;
!=	Compara si los dos operandos son diferentes	es_diferente = num1 != num2;
Y	Evalúa si ambas condiciones son verdaderas	condicion = (num1 > 0) Y (num2 < 10);
O	Evalúa si al menos una de las condiciones es verdadera	condicion = (num1 == 0) O (num2 == 0);
NO	Niega la condición	condicion = NO (num1 > 10);

Expresiones lógicas

Las expresiones lógicas son combinaciones de operadores relacionales y lógicos que evalúan condiciones y devuelven un valor verdadero o falso. Estas expresiones son fundamentales para controlar el flujo de ejecución de un programa mediante estructuras de control como el condicional "Si" o los bucles.

Por ejemplo, una expresión lógica en PSeInt podría ser:

```
es_mayor = edad > 18 Y sexo == 'M';
```

Esta expresión evaluará si la edad es mayor que 18 y si el sexo es masculino, devolviendo verdadero o falso según sea el caso.

Instrucciones Entrada / Salida

Las instrucciones de entrada y salida nos permiten interactuar con el usuario al recibir datos de entrada y mostrar resultados de salida. En PSeInt, utilizamos las instrucciones "Escribir" para mostrar información al usuario y "Leer" para obtener información del usuario.

Por ejemplo, para solicitar al usuario que ingrese su nombre y luego mostrar un mensaje de bienvenida, podríamos hacer lo siguiente:

```
Escribir("Por favor, ingrese su nombre: ");  
Leer(nombre);  
Escribir("¡Hola, ", nombre, "! Bienvenido al programa.");
```

Estas instrucciones son fundamentales para crear programas interactivos y amigables para el usuario en PSeInt.

En resumen, los operadores y expresiones nos permiten realizar cálculos, comparaciones y tomar decisiones en nuestros programas, mientras que las instrucciones de entrada y salida nos permiten interactuar con el usuario. Comprender y utilizar adecuadamente estos conceptos es esencial para desarrollar programas efectivos y funcionales en PSeInt.

Estructura general de un Algoritmo

En PSeInt, al igual que en otros lenguajes de programación, los algoritmos siguen una estructura específica que define cómo se organizan las instrucciones y datos dentro de un programa. En esta sección, exploraremos la estructura general de un algoritmo en PSeInt y cómo utilizarla para crear programas efectivos y funcionales.

Inicio del Algoritmo

El inicio del algoritmo marca el punto de entrada de nuestro programa. Aquí es donde comenzamos a escribir las instrucciones que formarán parte de nuestro algoritmo. La estructura básica de inicio de un algoritmo en PSeInt es la siguiente:

Algoritmo *titulo*

Declaración e Inicialización de Variables

Antes de comenzar a escribir el código de nuestro algoritmo, es importante declarar todas las variables que vamos a utilizar y asignarles un valor inicial si es necesario. Esto se hace para reservar espacio en la memoria y evitar errores al manipular los datos. Por ejemplo:

```
//Bloque de declaración
Definir total_sillas Como Entero;
Definir peso, altura Como Real;
Definir nombre Como Caracter;
Definir mayor_de_edad Como Logico;

//Bloque de Inicialización
total_sillas = 0;
peso = 0;
altura= 0;
nombre = "";
mayor_de_edad = FALSO;
```

Instrucciones del Algoritmo

Dentro del bloque de inicio, podemos incluir todas las instrucciones necesarias para realizar las tareas específicas de nuestro programa. Esto puede incluir operaciones aritméticas, decisiones condicionales, bucles de repetición, entrada y salida de datos, entre otros. Por ejemplo:

```
Algoritmo titulo
    Definir edad Como Entero;
    Definir precio Como Real;
    edad = 0;
    precio = 0;
    Escribir("Por favor, ingrese su edad: ");
    Leer edad;
    precio = edad * 10.5;
    Escribir "El precio es: ", precio;
FinAlgoritmo
```

El fin del algoritmo marca el final de nuestro programa. Aquí finalizamos todas las tareas y liberamos los recursos que ya no son necesarios. La estructura básica para el fin del algoritmo en PSeInt es la siguiente:

```
FinAlgoritmo
```

Estructuras Básicas

En PSeInt, las estructuras básicas y los comentarios son herramientas fundamentales que nos permiten organizar y documentar nuestros algoritmos de manera efectiva. En esta sección, exploraremos cómo utilizar estas estructuras para crear programas claros, legibles y fáciles de entender.

Los comentarios

Los comentarios son partes del código que no se ejecutan y sirven para proporcionar información adicional sobre el funcionamiento del programa. En PSeInt, los comentarios se escriben precedidos por el símbolo "//" y pueden estar en una línea individual o al final de una línea de código. Por ejemplo

```
// Este es un comentario de una línea
Edad = 0; // Este es un comentario al final de la línea
```

Los comentarios son útiles para explicar el propósito de ciertas partes del código, documentar el funcionamiento de un algoritmo y facilitar la comprensión para otros programadores que lean nuestro código en el futuro.

Bloque de asignación

El bloque de asignación nos permite asignar valores a las variables. En PSeInt, utilizamos el operador "=" para realizar asignaciones. Por ejemplo:

```
Algoritmo titulo
    Definir edad Como Entero;
    Definir precio Como Real;
    edad = 5;
    precio = 9;
FinAlgoritmo
```

La estructura secuencial

La estructura secuencial es la forma más básica de estructurar un algoritmo, donde las instrucciones se ejecutan en secuencia, una después de la otra, en el orden en que aparecen. Por ejemplo:

```
Algoritmo sin_titulo
    Escribir('Hola, ');
    Escribir "mundo!";
FinAlgoritmo
```

En este ejemplo, primero se muestra "Hola, " y luego se muestra "mundo!".

La estructura alternativa

La estructura alternativa nos permite tomar decisiones basadas en condiciones lógicas. En PSeInt, utilizamos la estructura "Si..Entonces..FinSi" para implementar esta estructura. Por ejemplo:

```
Algoritmo sin_titulo
    Definir edad Como Entero;
    edad = 0;
    Si edad >= 18 Entonces
        Escribir("Eres mayor de edad");
    FinSi
FinAlgoritmo
```

En este ejemplo, se muestra el mensaje "Eres mayor de edad" si la variable "edad" es mayor o igual a 18.

La estructura iterativa

La estructura iterativa nos permite repetir un bloque de código mientras se cumpla una condición. En PSeInt, utilizamos la estructura "Mientras..Hacer..FinMientras" para implementar bucles. Por ejemplo:

```
Algoritmo sin_titulo
    Definir contador Como Entero;
    Mientras contador < 5 Hacer
        Escribir "Contador: ", contador;
        contador = contador + 1;
    FinMientras
FinAlgoritmo
```

Bibliografía y Webgrafía

Steven S. Skiena, Miguel A. Revilla, et ál. Desafíos de programación: El manual de entrenamiento para concursos de programación (Spanish Edition) OJBooks (2020)

Vickler A. Algoritmos. Ladoo Publishing LLC (2022)

Pes C. Pseudocódigo para principiantes. Independently published (27 Abril 2022)

Glosario

- **Aritmético:** Relacionado con las operaciones matemáticas básicas como la suma, resta, multiplicación y división.
- **Constante:** Un valor que no cambia durante la ejecución del programa, como pi (π) o la velocidad de la luz.
- **Continuos:** Valores que pueden tomar cualquier valor dentro de un rango determinado, como la altura de una persona.
- **Cualitativo:** Se refiere a datos que describen cualidades o atributos que no pueden ser cuantificados numéricamente de forma directa.
- **Cuantitativo:** Relacionado con datos que representan cantidades numéricas, que pueden ser contables (discretos) o medibles (continuos).
- **Discretos:** Valores contables y separados por unidades definidas, como el número de estudiantes en un salón de clases.
- **Identificador:** Un nombre utilizado para identificar y hacer referencia a una variable, constante, función u otro elemento en un programa.
- **Interactivo:** Que permite la interacción con el usuario, usualmente a través de instrucciones de entrada y salida.
- **Lógico:** Relacionado con las operaciones booleanas, donde se evalúan condiciones y se devuelven valores de verdadero o falso.
- **Nominales:** Categorías o clases que no tienen un orden específico, como género o tipo de animal.

- **Ordinales:** Indican un orden o jerarquía entre las categorías, como niveles de satisfacción (bajo, medio, alto).
- **Relacional:** Referente a las relaciones o comparaciones entre valores, como mayor que ($>$), menor que ($<$), igual a ($=$), etc.
- **Residuo:** El resultado de una operación de módulo, que representa el resto de la división entera entre dos números.